

NetworkDevicesMonitor

Ivaniciuc Teodor-Arsenie

Universitatea "Alexandru Ioan Cuza" din Iasi, Facultatea de Informatica
Iasi, Romania

`teodor-arsenie.ivaniciuc@info.uaic.ro`

Abstract. NetworkDevicesMonitor este o platforma software completa pentru monitorizarea si analiza infrastructurii IT, integrand protocoale standardizate (Syslog) cu agenti personalizati pentru colectarea telemetriei. Arhitectura multi-threaded implementeaza pattern-uri avansate (Producer-Consumer, Thread-Safe Queues) si ofera o interfata grafica asincrona Qt6 cu dashboard-uri interactive, vizualizari în timp real, sistem de alertare automata si filtrare avansata. Proiectul demonstreaza aplicarea conceptelor de retele, concurenta, baze de date si design UI/UX profesional, incluzand functionalitati complete de securitate prin whitelist/blacklist, management alerte personalizate si filtre dinamice.

1 Introducere si Context

1.1 Motivatie

În infrastructurile IT moderne, echipamentele de retea (routere, switch-uri, firewall-uri) si statiile de lucru genereaza volume masive de date sub forma de log-uri si metrice. Monitorizarea manuala devine rapid impracticabila, conducand la:

- Pierderea evenimentelor critice de securitate
- Dificultati în diagnosticarea problemelor de performanta
- Lipsa vizibilitatii centralizate asupra infrastructurii
- Timp de raspuns crescut la incidente
- Absenta filtrarii inteligente a datelor relevante
- Management inefficient al alertelor si notificarilor

1.2 Obiective Tehnice

NetworkDevicesMonitor adreseaza aceste provocari prin:

1. Colectare Hibrida:

- Syslog (RFC 5424) pe portul 1514 pentru echipamente standard
- Agenti custom pe portul 9000 pentru metrice detaliate (CPU, RAM, disk)
- Protocol JUNK pe portul 8080 pentru comunicare client-server

2. Procesare Concurenta Performanta:

- I/O Multiplexing cu `poll()` pentru receptie non-blocanta

- Thread-Safe Queues pentru decuplarea ingestiei de procesare
 - Worker threads pentru parsing si analiza paralela
 - Shared mutex pentru operatiuni concurrent read-heavy
3. **Securitate prin Filtrare Multi-Nivel:**
 - Source Manager cu Whitelist/Blacklist IP-uri
 - Validare automata a surselor înainte de procesare
 - Izolare trafic malitios si unknown sources
 - Filtre personalizate pe tip, mesaj si alerte custom
 4. **Vizualizare si UX Avansat:**
 - Dashboard asincron Qt6 cu actualizari în timp real
 - Grafice interactive (Donut Charts, Line Charts)
 - Sistem de alertare pop-up pentru evenimente critice
 - Pagini dedicate pentru Syslog, Agents, Security si Filtres
 5. **Management Avansat:**
 - Sistem de filtrare dinamica (type, message, custom alerts)
 - Management centralizat al alertelor cu rezolvare
 - Separare date cunoscute vs. unknown sources

2 Arhitectura Sistemului

2.1 Vedere de Ansamblu

Sistemul este structurat în 3 componente majore care comunica prin protocoale TCP:

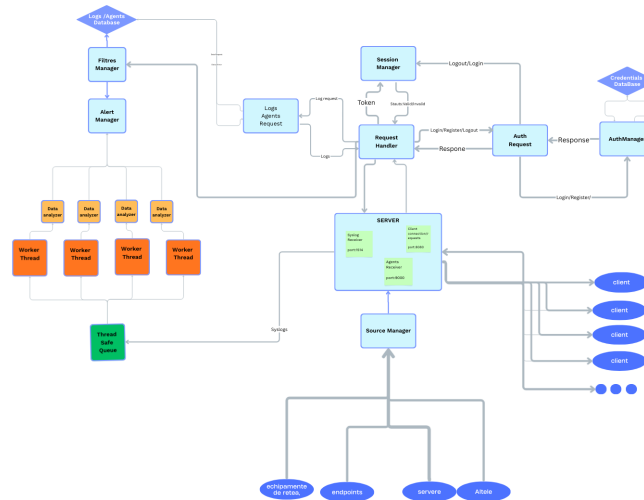


Fig. 1: Arhitectura completa NetworkDevicesMonitor

1. Server Backend (C++23) Inima sistemului, responsabila pentru:

- Receptia si procesarea datelor de pe 3 canale TCP simultane
- Management baze de date SQLite3 cu WAL mode pentru concurenta
- Autentificare, autorizare si management sesiuni cu token-based auth
- Detectare si notificare alerte bazate pe pattern-uri
- Filtrare multi-nivel: source IP, tip mesaj, keyword matching
- Procesare paralela cu thread pools

Componente principale:

- LogPipeline - Orchestrare flux de date
- SyslogReceiver - Port 1514, RFC 5424 parsing
- AgentsReceiver - Port 9000, JSON metrics
- ConnectionServer - Port 8080, Admin API
- DataProcessor - Analiza si stocare
- SourceManager - Whitelist/Blacklist
- AlertsManager - Detectare si management alerte
- FiltresManager - Sistem de filtrare avansata
- DBManager - Thread-safe database operations

2. Client GUI (Qt6) Interfata grafica profesionala cu:

- Sistem de pagini modular cu lifecycle hooks (on_enter/on_exit)
- Comunicare asincrona (Qt Signals & Slots)
- Vizualizari tabelare
- Grafice în timp real cu QtCharts
- Popup-uri pentru alerte

Pagini implementate:

1. **ConnectPage** - Conectare la server (IP + port)
2. **LoginPage** - Autentificare utilizator
3. **HomePage** - Dashboard principal cu tiles de navigare
4. **DashboardPage** - 4 sub-pagini:
 - Syslog Dashboard (tabele + donut/line charts)
 - Agents Dashboard (metrice în timp real)
 - Unknown Syslog Data (trafic neautorizat)
 - Unknown Agent Data (agenti necunoscuti)
5. **SecurityPage** - Management Whitelist/Blacklist cu add/remove
6. **AlertsPage** - Vizualizare si rezolvare alerte
7. **FiltresPage** - Configurare filtre (type, message, custom alerts)
8. **SettingsPage** - Configurari aplicatie

2.2 Pipeline-ul de Date

Port 1514 - Syslog Receiver Fluxul de procesare:

1. **Receptie:** Thread dedicat cu `poll()` monitorizeaza socket-ul
2. **Validare IP:** Source Manager verifica whitelist/blacklist
3. **Parsare:** Extragere campuri conform RFC 5424:
 - PRI (Priority = Facility $\times$ 8 + Severity)
 - Timestamp
 - Hostname
 - Source (proces/serviciu)
 - Message content
4. **Filtrare:** FiltresManager aplica reguli (type, message keywords)
5. **Analiza:** AlertsManager detecteaza pattern-uri suspecte
6. **Stocare:** Insert în logs (known) sau `unknown_log`

Exemplu mesaj Syslog procesat:

```
<13>1 2025-11-27T15:37:44Z server02 sshd 3452 - -
Accepted password for user_valid from 192.168.2.5
```

Implementare în cod:

```
// server/src/log_pipeline/syslog_server/SyslogReceiver.cpp
void SyslogReceiver::start() {
    std::vector<pollfd> fds;
    pollfd server_poll = {server_fd. value(), POLLIN, 0};
    fds.push_back(server_poll);

    while(true) {
        poll(fds.data(), fds.size(), -1);
        for (auto& fd : fds) {
            if (!(fd.revents & POLLIN)) continue;

            if (fd.fd == server_fd.value()) {
                // Accept new connection
                int client_fd = accept(... );
                std::string ip = get_client_ip(client_fd);

                // Security check
                if(source_manager->check_ip_blacklist(ip)) {
                    close(client_fd);
                    continue;
                }

                client_ips[client_fd] = ip;
                fds.push_back({client_fd, POLLIN, 0});
            } else {
                // Read data from existing client
                RW_logs(fd.fd);
            }
        }
    }
}
```

```

    }
  }
}

```

Port 9000 - Agent Receiver Protocol de comunicare:

- Header binar: 4 bytes = lungime payload
- Payload JSON cu metrice

```

{
  "hostname": "workstation-01",
  "ip_address": "192.168.1.10",
  "os_info": "Linux",
  "cpu_load": 45.2,
  "ram_usage": 68.5,
  "disk_usage": 72.1,
  "status_message": "OK",
  "active_user": "admin"
}

```

Port 8080 - Admin API Protocol custom JUNK (Just Useful Notation Kinda):

Format: key:{value};key2:{value2};

Exemplu cerere login:

```
type:{login};username:{admin};password:{pass123};
```

Exemplu raspuns:

```
succes:{true};type:{login};content:{token_abc123xyz};
```

Implementare JUNK Parser:

```

// server/src/utils/JUNK. cpp
std::expected<JUNK, std::string> JUNK::deserialize(std::
string data) {
    JUNK new_data;
    int left_count = 0, format_count = 0, poz = 0;
    std::string key_val[2];

    for(auto ch : data) {
        if (ch == '\n' || ch == '\r') continue;

        if(ch == ':' && left_count == 0) {
            poz = 1;
            format_count++;
            continue;

```

```

    }
    if(ch == '{') {
        left_count++;
        if (left_count == 1) continue;
    }
    if(ch == '}') {
        left_count--;
        if (left_count == 0) continue;
    }
    if(ch == ';' && left_count == 0) {
        new_data.add(key_val[0], key_val[1]);
        poz = 0;
        key_val[0].clear();
        key_val[1].clear();
        continue;
    }
    key_val[poz].push_back(ch);
}

if(format_count == 0 || left_count != 0 || new_data.empty())
    return std::unexpected("Invalid JUNK format");
return new_data;
}

```

3 Implementare Tehnica Detaliata

3.1 Server: Concurenta si Thread Safety

Thread-Safe Queue Implementation Pattern Producer-Consumer pentru decuplarea I/O de procesare:

```

// server/src/utils/ThreadSafeQueue.hpp
template<class T>
class ThreadSafeQueue {
    std::queue<T> queue;
    std::mutex _mutex;

public:
    void push(T value) {
        std::lock_guard<std::mutex> lock(_mutex);
        this->queue.push(value);
    }

    std::expected<T, std::string> pop() {
        std::lock_guard<std::mutex> lock(_mutex);
        if(queue.empty())
            return std::unexpected("Empty queue");
        auto value = queue.front();
    }
}

```

```

        queue.pop();
        return value;
    }
};

// Event structure
enum EventType { SYSLOG, AGENT_METRIC };
struct LogEvent {
    EventType type;
    std::string source_ip;
    std::string payload;
};

```

Database Manager (SQLite3 WAL Mode) Caracteristici cheie:

- shared_mutex pentru Read-Write locking concurrent
- Prepared statements pentru securitate anti-SQL injection
- WAL (Write-Ahead Logging) pentru citiri simultane

Tabele principale:

1. logs - Syslog-uri de la surse cunoscute (whitelist)
2. unknown_log - Trafic de la surse necunoscute
3. agents_metrics - Date de la agenti de monitorizare
4. users - Credentiale administratori (token-based auth)
5. whitelist - Surse validate (IP + hostname)
6. blacklist - Surse blocate
7. alerts - Evenimente critice detectate
8. filters_type - Filtre pe tip de log
9. filters_message - Filtre pe keyword în mesaj
10. filters_alerts - Alerte personalizate

Source Manager - Sistem de Securitate Operatiuni principale:

```

// server/src/log_pipeline/source_managers/SourceManager.hpp
class SourceManager {
    std::shared_mutex _mutex;
    std::unordered_map<std::string, std::string>
        whitelist_source; // IP -> Hostname
    std::unordered_set<std::string> blacklist_source;
    std::shared_ptr<DBManager> db;

public:
    std::optional<std::string> check_ip_whitelist(std::string
        ip) {
        std::shared_lock lock(_mutex);
        if(whitelist_source.contains(ip))
            return whitelist_source[ip];
    }
};

```

```

        return std::nullopt;
    }

    bool check_ip_blacklist(std::string ip) {
        std::shared_lock lock(_mutex);
        return blacklist_source.contains(ip);
    }

    void add_to_whitelist(std::string ip, std::string
        hostname) {
        std::unique_lock lock(_mutex);
        whitelist_source[ip] = hostname;
        // Persist to DB
        db->query("INSERT INTO whitelist VALUES (?, ?)", {ip,
            hostname});
    }

    void add_to_blacklist(std::string ip) {
        std::unique_lock lock(_mutex);
        blacklist_source.insert(ip);
        db->query("INSERT INTO blacklist VALUES (?)", {ip});
    }
};

```

3.2 Sistem de Filtrare Avansata

FiltresManager - Filtrare Multi-Nivel Sistemul implementeaza 3 tipuri de filtre:

1. Filtre pe Tip (Type Filters):

- Filtreaza log-uri dupa tipul evenimentului
- Matching exact pe campul **severity**
- Utilizare: exclude log-uri non-critice

2. Filtre pe Mesaj (Message Filters):

- Keyword matching în corpul mesajului
- Case-insensitive search
- Utilizare: cauta pattern-uri specifice (error, failed, denied)

3. Alerte Personalizate (Custom Alerts):

- Declansare alerte pe keyword-uri critice
- Notificare instant catre clienti conectati
- Severitate configurabila

3.3 Sistem de Alertare

AlertsManager (Server) Detectare pattern-uri suspecte:

- Tentative repetate de login (sper ca am terminat asta)
- Trafic de pe IP-uri din blacklist
- Consum CPU/RAM peste threshold-uri (>90% pentru >30 sec)
- Keyword matching din filtre custom

AlertsPage si AlertPopup (Client) AlertsPage - Management centralizat:

- Tabel cu toate alertele (ID, Type, Severity, Source, Message, Timestamp)
- Buton "Resolve" pentru marcarea alertelor ca rezolvate
- Auto-refresh la interval de 2 secunde

AlertPopup - Notificari non-intruzive:

```
// client/src/frontend/widgets/AlertPopup.cpp
class AlertPopup : public QWidget {
public:
    void showMessage(QString title, QString message, bool
isCritical) {
        titleLabel->setText(title);
        messageLabel->setText(message);

        if(isCritical) {
            setStyleSheet(R"(
                QWidget {
                    background-color: #2C1B1B;
                    border-left: 5px solid #FF5555;
                }
            )");
        } else {
            setStyleSheet(R"(
                QWidget {
                    background-color: #2C2416;
                    border-left: 5px solid #F1C40F;
                }
            )");
        }
    }

    // Show in bottom-right corner
    QScreen *screen = QApplication::primaryScreen();
    QRect screenGeometry = screen->geometry();
    int x = screenGeometry.width() - this->width() - 20;
    int y = screenGeometry.height() - this->height() -
        20;
    this->move(x, y);
}
```

```

        this->show();

        // Auto-hide after 10 seconds
        QTimer::singleShot(10000, this, &AlertPopup::hide);
    }
};

```

Integrare cu DataRequester:

```

// client/src/frontend/app. cpp
connect(data_requester.get(), &DataRequester::
    UpdateAlertsPopup,
    this, [this](QString data) {
        auto junk = JUNK::deserialize(data.toStdString());
        if(!junk.has_value() || junk.value()["succes"].value
            () == "false")
            return;

        auto alerts_str = junk.value()["content"].value();
        auto alerts = parse_alerts(alerts_str);

        for(const auto& alert : alerts) {
            if(alert.id > last_alert_id) {
                bool isCritical = (alert.severity == "high"
                    ||
                                alert.severity == "critical
                                ");
                alert_popup->showMessage(
                    QString::fromStdString(alert.type),
                    QString::fromStdString(alert.message),
                    isCritical
                );
                last_alert_id = alert.id;
            }
        }
    });

```

3.4 Client: Interfata Grafica Asincrona

Page Management System Arhitectura modulara pentru navigare cu lifecycle management:

```

// client/src/frontend/page_system/PageManager.h
class PageManager {
    std::shared_ptr<QStackedWidget> stack;
    std::vector<std::shared_ptr<Page>> pages;
    std::shared_ptr<Page> current_page;

public:

```

```

void change_page(int index) {
    if(current_page)
        current_page->on_exit(); // Cleanup

    stack->setCurrentIndex(index);
    current_page = pages->at(index);
    current_page->on_enter(); // Initialize
}

void add_page(QWidget* page_widget) {
    stack->addWidget(page_widget);
}

};

// Abstract base class
class Page {
protected:
    QWidget* page;
    std::shared_ptr<DataRequester> data_requester;
    std::shared_ptr<PageManager> page_manager;
    std::shared_ptr<QMainWindow> window;

public:
    virtual void on_enter() = 0; // Called when page becomes
        active
    virtual void on_exit() = 0; // Called when leaving page
    QWidget* get_page() { return page; }
};

```

Comunicare Asincrona cu Signals

```

// client/src/server_request/DataRequester.h
class DataRequester : public QObject {
    Q_OBJECT

signals:
    void LoginData(QString data);
    void LogsData(QString data);
    void UpdateAlertsDashboard(QString response);
    void UpdateWhitelist(QString response);
    void UpdateBlacklist(QString response);
    void UpdateAlertsPopup(QString response);
    void UpdateSyslogDashboard(QString response);
    void UpdateAgentsDashboard(QString response);
    void UpdateUnknownSyslog(QString response);
    void UpdateUnknownAgent(QString response);
    void UpdateFiltres(QString response);
    void lost_connection();

public slots:

```

```

void start_receiving() {
    while(connected) {
        auto packet = receive();
        if(! packet.has_value()) {
            emit lost_connection();
            break;
        }

        auto junk = JUNK::deserialize(packet.value());
        if(! junk.has_value()) continue;

        std::string type = junk.value()["type"].value_or(
            "");

        if(type == "login")
            emit LoginData(QString::fromStdString(packet.
                value()));
        else if(type == "logs")
            emit LogsData(QString::fromStdString(packet.
                value()));
        else if(type == "alerts")
            emit UpdateAlertsDashboard(QString::
                fromStdString(packet.value()));
        // ... etc pentru toate tipurile
    }
}
};

```

Conectare în UI:

```

// client/src/frontend/app.cpp
connect(data_requester.get(), &DataRequester::LogsData,
    this, [this](QString data) {
        auto junk = JUNK::deserialize(data.toStdString());
        if(!junk.has_value()) return;

        auto logs_str = junk.value()["content"].value();
        auto logs = BetterString::split(logs_str, " : ;");

        // Update table
        syslog_dashboard->update_table(logs);

        // Update charts
        syslog_dashboard->update_charts(logs);
    });

```

Security Page - Whitelist/Blacklist Management Functionalitati:

- 2 tab-uri: Whitelist si Blacklist

- Tabel cu IP si Hostname (whitelist) sau doar IP (blacklist)
- Buton "Add" deschide popup modal
- Delete la click pe rand în tabel
- Sincronizare instant cu server
- Conflict detection (IP deja în cealalta lista)

4 Protocol de Comunicare

4.1 Structura Pachetului (Admin API)

Bytes	Tip	Descriere
0-3	int	Lungime payload
4-N	ASCII Text	Date în format JUNK

Table 1: Format pachet TCP pe port 8080

4.2 Comenzi Disponibile

Comanda	Format cerere
Login	type:{login};username:{...};password:{...};
Register	type:{register};username:{...};password:{...};
Get Logs	type:{logs};last_id:{123};token:{...};
Get Agents	type:{agents};last_id:{0};token:{...};
Add Whitelist	type:{add_whitelist};ip:{...};hostname:{...};token:{...};
Add Blacklist	type:{add_blacklist};ip:{...};token:{...};
Remove Whitelist	type:{remove_whitelist};ip:{...};token:{...};
Remove Blacklist	type:{remove_blacklist};ip:{...};token:{...};
Get Alerts	type:{update_alerts};last_id:{0};token:{...};
Resolve Alert	type:{remove_alert};alert_id:{123};token:{...};
Add Type Filter	type:{add_type_filter};keyword:{SSH};token:{...};
Add Msg Filter	type:{add_msg_filter};keyword:{error};token:{...};
Add Custom Alert	type:{add_custom_alert};keyword:{critical};token:{...};
Get Filtres	type:{get_filtres};token:{...};
Unknown Syslog	type:{unknown_syslog};last_id:{0};token:{...};
Unknown Agents	type:{unknown_agents};last_id:{0};token:{...};

Table 2: API endpoints complete

4.3 Exemple de Raspunsuri

Raspuns succes login:

```
succes:{true};type:{login};content:{
    eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9};
```

Raspuns logs:

```
succes:{true};type:{logs};content:{
    1[[]2025-11-27 15:30:00[[]4[[] server01[[] sshd[[] Login
      successful: ;;
    2[[]2025-11-27 15:31:00[[]3[[] router01[[] firewall[[] Blocked
      connection:;;
};
```

Raspuns eroare:

```
succes:{false};type:{error};content:{Invalid token};
```

4.4 Validare cu Wireshark

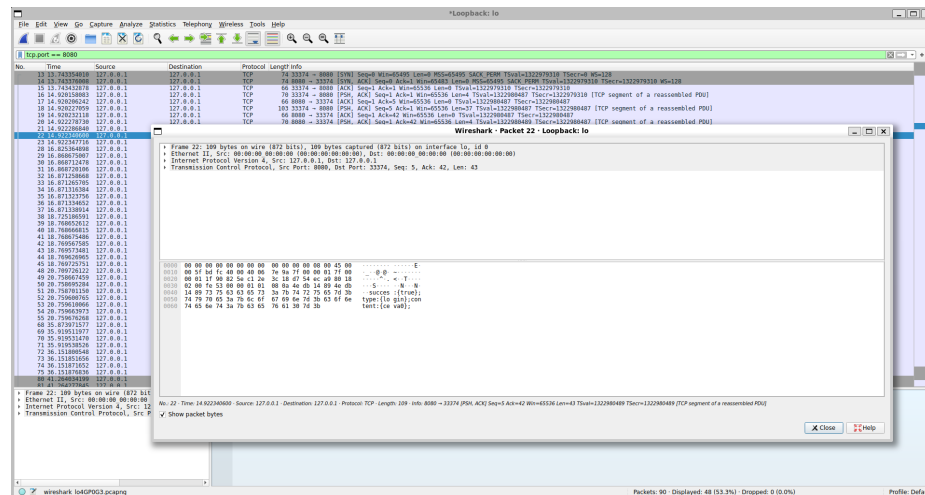


Fig. 2: Captura Wireshark - Analiza pachetelor JUNK

5 Scenarii de Utilizare

5.1 Scenarii de Succes

Scenariu 1: Monitorizare Agent în Timp Real Context: Un server de baza de date PostgreSQL trimite metrice la interval de 5 secunde.

Flow:

1. Agent Python colecteaza date cu psutil:

```
cpu_load = psutil.cpu_percent(interval=1)
ram_usage = psutil.virtual_memory().percent
disk_usage = psutil.disk_usage('/').percent
```

2. Pachet JSON trimis pe port 9000 cu header binar
3. Server primește, validează IP în whitelist via SourceManager
4. DataProcessor extrage metrice, insert în `agents_metrics`
5. Client primește signal `AgentDataUpdated`
6. Dashboard actualizează grafic CPU/RAM automat fără refresh

Rezultat: Administrator vizualizează în sub 3 secunda creșterea CPU la 92%, investigare preventivă înainte de crash.

Scenariu 2: Onboarding Echipament Nou Flow:

1. Admin accesează SecurityPage → Whitelist tab
2. Click "Add Source", completează în popup:
 - IP: 192.168.10.50
 - Hostname: switch-floor2
3. Click "Add" → cerere JUNK trimisă la server:

```
type:{add_whitelist};ip:{192.168.10.50};hostname:{switch-floor2};
```

4. SourceManager inserează în DB + hash map in-memory
5. Switch-ul configurează syslog destination către server: 1514
6. Primul syslog primit, IP verificat în whitelist → ACCEPTED
7. Procesat imediat, apare în Syslog Dashboard cu label "switch-floor2"
8. Donut chart actualizat cu noi date

Rezultat: Zero downtime, instant visibility pentru noul device.

Scenariu 3: Configurare Filtre Custom Context: Admin vrea să ignore log-uri verbose de la servere web (HTTP 200 OK).

Flow:

1. Accesează FiltresPage → Message Filters tab
2. Click "Add Message Filter"
3. Introduce keyword: "HTTP 200"
4. Server adaugă în `filtres_message` table
5. FiltresManager încarcă în memory cache
6. Log-uri următoare cu "HTTP 200" sunt filtrate automat
7. Dashboard rămâne curat cu doar evenimente relevante

Rezultat: Reducere zgomotul în dashboard, focus pe evenimente critice.

Scenariu 4: Alerta Personalizata pe Keyword Context: Admin vrea notifica instant la orice mesaj continand "OutOfMemory".

Flow:

1. FiltresPage → Custom Alerts tab → "Add Custom Alert"
2. Introducere keyword: "OutOfMemory"
3. Server adauga în `filtres_alerts`
4. Un serviciu Java crasheaza cu "java.lang.OutOfMemoryError"
5. Log syslog primit, `DataProcessor` match keyword
6. `AlertsManager` creeaza alerta CRITICAL
7. Popup instant la client: "OutOfMemory error detected on app-server-03"
8. Admin verifica `AlertsPage`, vede stack trace complet

Rezultat: Detectie proactiva, restart serviciu înainte de afectare utilizatori.

5.2 Scenarii de Esec si Recuperare

Scenariu 5: Trafic Malitios Blocat Context: Botnet încearca flood de date false pe port 1514.

Flow:

1. 1000 de pachete/sec de pe IP 198.51.100.10 (în blacklist)
2. `SyslogReceiver` detecteaza conexiune în `accept()`
3. `SourceManager` verifica: `check_ip_blacklist()` → `true`
4. Socket închis imediat cu `close(client_fd)`, zero processing
5. Log în `security_events`: "Blocked blacklist IP 198.51.100.10"
6. Statistici actualizate: "X blocked attempts today"

Rezultat: Zero impact pe performanta, CPU usage normal (<5%).

Scenariu 6: Pachet Corupt Primit Context: Eroare de retea corupe header-ul unui pachet JUNK.

Flow:

1. Client trimite pachet cu lungime incorecta în header (0xFFFFFFFF)
2. Server citește 4 bytes: `length = 4294967295` (invalid)
3. Buffer allocation failsafe: maxim 64KB hardcoded
4. Alloc doar 64KB, citire partiala
5. `JUNK::deserialize()` returneaza `std::unexpected("Invalid format")`
6. `RequestHandler` logheaza eroare, trimite:

```
succes:{false};type:{error};content:{Invalid JUNK format};
```

7. Client afiseaza `QMessageBox`: "Request failed, please retry"

Rezultat: Server nu crasheaza, conexiune mentinuta, utilizator poate re-trimite.

Scenariu 7: Token Expirat Context: Admin lasa aplicatia deschisa 24h, sesiune expirata pe server.

Flow:

1. Client încearca sa ceara logs cu token vechi
2. SessionManager verifica: token nu mai exista în hash map (cleared dupa 12h)
3. Raspuns: `succes:{false};type:{error};content:{Session expired};`

Rezultat: Security enforcement, experienta user fluida.

6 Concluzii

Acest proiect a fost, înainte de toate, o experienta de învățare foarte utila. Daca la început totul era doar o schita, acum am un server functional (cu Syslog si Agenti) si un client care chiar comunica între ei.

Ce am învățat pe parcurs

Nu a fost doar despre scris cod, ci despre a înțelege cum se leaga lucrurile în prezent:

- **C++23:** Am profitat de ocazie sa explorez noutatile din noul standard pentru a scrie codul mai modern.
- **Concurenta:** Am învățat sa gestionez mai bine firele de executie.
- **Arhitectura:** Am pus mare pret pe modularitate. Faptul ca am împartit totul pe clase si module m-a ajutat enorm sa nu ma pierd în detalii pe masura ce proiectul crestea.

O privire realista

Sunt constient ca proiectul nu este perfect si ca, daca as mai avea timp, ar mai fi destule „colturi de slefuit” sau functionalitati de optimizat. Totusi, pentru stadiul actual, consider ca a iesit un proiect bun, stabil si, cel mai important, usor de înțeles datorita structurii sale ordonate.

A fost o provocare care mi-a aratat exact unde mai am de lucrat, dar si cat de mult am progresat.

References

1. Facultatea de Informatica Iasi: Retele de Calculatoare - Resurse Curs si Laborator. <https://edu.info.uaic.ro/computer-networks/cursullaboratorul.php>
2. GeeksforGeeks: Multithreading in C++. <https://www.geeksforgeeks.org/cpp/multithreading-in-cpp/>
3. C++ Reference: std::expected (Error handling). <https://en.cppreference.com/w/cpp/utility/expected.html>

4. Splunk: What is Syslog? A Complete Guide to Syslog Monitoring. https://www.splunk.com/en_us/blog/learn/syslog.html
5. Qt Documentation <https://doc.qt.io/>
6. Json Documentation <https://json.nlohmann.me/>
7. Sqlite <https://sqlite.org/cli.html>